

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Music Streaming Platform**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- Users can register and log in with free or premium tier accounts.
- Users can search for songs by title, artist name, album, or genre.
- Users can play songs, albums, and playlists using a unified playback interface.
- Users can control playback: play, pause, resume, and skip to next/previous track.
- Users can create, update, and delete playlists, and add or remove songs.
- The system should support minimal initial buffering for fast playback.
- Users can listen to downloaded songs offline.
- Premium users get ad-free listening, unlimited skips, and offline downloads.
- Free users have limited skips, standard audio quality, and advertisements.
- The system should provide personalized song recommendations.

Interview Tip: Clarify user tiers, playback guarantees, and offline behavior early to avoid design rework later.

Counter Questions to Ask Interviewer

- Is there a device limit for premium users?
 - Which streaming protocol should be used?
 - Should audio quality be adaptive or fixed based on network conditions?
 - Can playlists be shared, and who has edit/delete permissions?
 - Which audio formats are supported?
 - Is there an API to upgrade users from free to premium?
 - Can a song belong to multiple albums or playlists?
 - Should playback state sync across multiple devices?
-

Step 2: Identify Core Entities

1. Song (Core Entity)

- id: int [PK]
- songId: String [UNIQUE]
- title: String
- artistId: int [FK to Artist]
- albumId: int [FK to Album, NULLABLE]
- duration: long (in seconds)
- genre: String
- audioUrl: String (URL to audio file on CDN/storage)

- thumbnailUrl: String (URL to album art/thumbnail)
- fileSize: long (in bytes)
- quality: AudioQuality (Enum: STANDARD, HIGH, PREMIUM)
- format: AudioFormat (Enum: MP3, AAC, WAV)
- createdAt: LocalDateTime

2. User

- id: int [PK]
- username: String [UNIQUE]
- email: String [UNIQUE]
- name: String
- subscriptionTier: SubscriptionTier (Enum: FREE, PREMIUM)
- createdAt: LocalDateTime

3. Playlist

- id: int [PK]
- playlistId: String [UNIQUE]
- name: String
- userId: int [FK to User]
- isPublic: boolean (false = private, true = public/shared)
- songIds: List<String> (ordered list of song IDs)
- createdAt: LocalDateTime
- updatedAt: LocalDateTime

4. Album

- id: int [PK]
- albumId: String [UNIQUE]
- title: String
- artistId: int [FK to Artist]
- releaseDate: LocalDate
- thumbnailUrl: String
- createdAt: LocalDateTime

5. Artist

- id: int [PK]
- artistId: String [UNIQUE]
- name: String
- thumbnailUrl: String
- createdAt: LocalDateTime

6. PlaybackSession (Core Entity)

- id: int [PK]
- sessionId: String [UNIQUE]
- userId: int [FK to User]
- currentSongId: String [FK to Song]
- currentPosition: long (playback position in seconds)
- playbackSource: PlaybackSource (Enum: SONG, ALBUM, PLAYLIST)
- sourceId: String (songId, albumId, or playlistId)
- queue: List<String> (ordered list of song IDs)
- shuffleMode: boolean
- repeatMode: RepeatMode (Enum: OFF, ONE, ALL)
- status: PlaybackStatus (Enum: PLAYING, PAUSED, STOPPED)
- deviceId: String (device identifier for multi-device sync)
- startedAt: LocalDateTime
- lastUpdatedAt: LocalDateTime

7. Download (Core Entity – Offline Playback)

- id: int [PK]
- downloadId: String [UNIQUE]
- userId: int [FK to User]
- songId: String [FK to Song]
- deviceId: String
- downloadStatus: DownloadStatus (Enum: PENDING, IN_PROGRESS, COMPLETED, FAILED)
- localFilePath: String [NULLABLE]
- downloadedAt: LocalDateTime [NULLABLE]
- createdAt: LocalDateTime

8. ListeningHistory

- id: int [PK]
- userId: int [FK to User]
- songId: String [FK to Song]
- playedAt: LocalDateTime
- playDuration: long (seconds listened)
- completed: boolean

9. Enums

- SubscriptionTier: FREE, PREMIUM
- AudioQuality: STANDARD (128kbps), HIGH (256kbps), PREMIUM (320kbps)
- AudioFormat: MP3, AAC, WAV
- PlaybackSource: SONG, ALBUM, PLAYLIST
- RepeatMode: OFF, ONE, ALL
- PlaybackStatus: PLAYING, PAUSED, STOPPED
- DownloadStatus: PENDING, IN_PROGRESS, COMPLETED, FAILED

10. Response DTOs (Not Stored)

- **SearchResponse:** songs, albums, artists, playlists
- **PlaybackStateResponse:** sessionId, currentSong, queue, position, status
- **RecommendationResponse:** recommended songs list

11. Request DTOs

- PlayRequest
- PlaylistRequest
- UpdatePositionRequest
- SearchRequest
- DownloadRequest

Interview Tip: Identifying playback sessions, downloads, and listening history early helps reason about latency, offline support, recommendations, and multi-device sync.

Step 3: Visualize Interaction Flows

1. Search Songs:

- GET /api/search?query={query}&type={type} → SearchController.search(request)
- → SearchService.search(query, type)
- → SongRepository.findByTitle / Artist / Album(query)
- → Build SearchResponse
- → Return

2. Play Song / Album / Playlist:

- POST /api/playback/play → PlaybackController.play(request)
- → PlaybackService.play(request)
- → Validate user subscription tier
- → Build queue from source (song / album / playlist)
- → Create or Update PlaybackSession (status=PLAYING)
- → StreamingService.getStreamUrl(songId, quality)
- → Return stream URL + PlaybackStateResponse

3. Stream Audio (Chunk-based):

- GET /api/stream/{songId}?start={byteOffset}&end={byteEnd}
- → StreamingController.stream(songId, start, end)
- → StreamingService.getChunk(songId, start, end)
- → CacheService.getChunk(songId, start, end)
- → If cached → return from cache
- → Else fetch from CDN/storage
- → CacheService.putChunk(songId, start, end, chunkData)

- → Return HTTP 206 Partial Content

4. Control Playback:

- POST /api/playback/pause → PlaybackController.pause(sessionId)
- → PlaybackService.pause(sessionId)
- → Update PlaybackSession(status=PAUSED, currentPosition)
- → Return PlaybackStateResponse
- POST /api/playback/resume → PlaybackController.resume(sessionId)
- → PlaybackService.resume(sessionId)
- → Update PlaybackSession(status=PLAYING)
- → Return PlaybackStateResponse
- POST /api/playback/skip-next → PlaybackController.skipNext(sessionId)
- → PlaybackService.skipNext(sessionId)
- → Get next song from queue (shuffle/repeat aware)
- → Update PlaybackSession(currentSongId, currentPosition=0)
- → Return PlaybackStateResponse

5. Create Playlist:

- POST /api/playlists → PlaylistController.createPlaylist(request)
- → PlaylistService.createPlaylist(userId, name, songIds)
- → Validate songs exist
- → Create Playlist
- → Return Playlist

6. Update Playlist:

- PUT /api/playlists/{playlistId} → PlaylistController.updatePlaylist(playlistId, request)
- → PlaylistService.updatePlaylist(playlistId, userId, name, songIds)
- → Acquire lock (playlist_lock_{playlistId})
- → Validate playlist ownership
- → Update playlist and updatedAt

- → Release lock
- → Return Playlist

7. Download Song (Offline Playback):

- POST /api/downloads → DownloadController.download(request)
- → DownloadService.download(userId, songId, deviceId)
- → Validate premium subscription
- → Validate device & download limits
- → Create Download(status=PENDING)
- → Async: StreamingService.downloadFullSong(songId)
- → Store in device cache
- → Update Download(status=COMPLETED, localFilePath)
- → Return Download

8. Get Recommendations:

- GET /api/recommendations → RecommendationController.getRecommendations(userId)
- → RecommendationService.getRecommendations(userId)
- → Fetch listening history
- → RecommendationStrategy.generate(userId, history)
- → Return RecommendationResponse

9. Get Playback State:

- GET /api/playback/state → PlaybackController.getState(sessionId)
- → PlaybackService.getState(sessionId)
- → Fetch PlaybackSession
- → Build PlaybackStateResponse
- → Return (client polls for sync)

10. Toggle Shuffle / Repeat:

- POST /api/playback/shuffle → PlaybackController.toggleShuffle(sessionId, enabled)
- → PlaybackService.toggleShuffle(sessionId, enabled)

- → Update PlaybackSession(shuffleMode)
- → Return PlaybackStateResponse

11. Update Playback Position (Periodic):

- POST /api/playback/position →
PlaybackController.updatePosition(sessionId, position)
 - → PlaybackService.updatePosition(sessionId, position)
 - → Update PlaybackSession(currentPosition, lastUpdatedAt)
 - → If $\geq 90\%$ played → mark completed
 - → Save ListeningHistory
 - → Return success (called every 30–60 seconds)
-

Step 4: Define Class Structures & Relationships

Layers of Architecture

We design the system using a clean **layered architecture** to ensure **separation of concerns**, **scalability**, and **testability**.

Client/UI → Controller Layer → Service Layer → Domain Layer

CONTROLLERS:

1. PlaybackController

- - PlaybackStateResponse play(PlayRequest request)
- - PlaybackStateResponse pause(String sessionId)
- - PlaybackStateResponse resume(String sessionId)

- - PlaybackStateResponse skipNext(String sessionId)
- - PlaybackStateResponse skipPrevious(String sessionId)
- - PlaybackStateResponse getState(String sessionId)
- - PlaybackStateResponse toggleShuffle(String sessionId, boolean enabled)
- - PlaybackStateResponse setRepeatMode(String sessionId, RepeatMode mode)
- - void updatePosition(String sessionId, long position)

2. SearchController

- - SearchResponse search(SearchRequest request)

3. PlaylistController

- - Playlist createPlaylist(PlaylistRequest request)
- - Playlist updatePlaylist(String playlistId, PlaylistRequest request)
- - void deletePlaylist(String playlistId, int userId)
- - Playlist addSongs(String playlistId, List<String> songIds, int userId)
- - Playlist removeSongs(String playlistId, List<String> songIds, int userId)

4. StreamingController

- - ResponseEntity<byte[]> stream(String songId, long start, long end, int userId)

5. DownloadController

- - Download download(DownloadRequest request)
- - List<Download> getDownloads(int userId, String deviceId)
- - void deleteDownload(String downloadId, int userId)

6. RecommendationController

- - RecommendationResponse getRecommendations(int userId)
-

SERVICES:

1. PlaybackService

- - PlaybackStateResponse play(PlayRequest request)
- - PlaybackStateResponse pause(String sessionId)
- - PlaybackStateResponse resume(String sessionId)
- - PlaybackStateResponse skipNext(String sessionId)
- - PlaybackStateResponse skipPrevious(String sessionId)
- - PlaybackStateResponse getState(String sessionId)
- - void updatePosition(String sessionId, long position)
- - void saveListeningHistory(int userId, String songId, long playDuration, boolean completed)

2. StreamingService

- - String getStreamUrl(String songId, AudioQuality quality)
- - byte[] getChunk(String songId, long start, long end)
- - void downloadFullSong(String songId, String deviceId)

3. CacheService

- - Optional<byte[]> getChunk(String songId, long start, long end)
- - void putChunk(String songId, long start, long end, byte[] chunk)
- - void evictChunk(String songId, long start, long end)
- - void evictSong(String songId)
- - void evictLRU(int maxSize)

4. PlaylistService

- - Playlist createPlaylist(int userId, String name, List<String> songIds)
- - Playlist updatePlaylist(String playlistId, int userId, String name, List<String> songIds)
- - void deletePlaylist(String playlistId, int userId)
- - Playlist addSongs(String playlistId, int userId, List<String> songIds)
- - Playlist removeSongs(String playlistId, int userId, List<String> songIds)

5. DownloadService

- - Download download(int userId, String songId, String deviceId)
- - List<Download> getDownloads(int userId, String deviceId)
- - void deleteDownload(String downloadId, int userId)
- - boolean validateDeviceLimit(int userId, String deviceId)
- - boolean validateDownloadLimit(int userId)

6. SearchService

- - SearchResponse search(String query, String type)

7. RecommendationService

- - RecommendationResponse getRecommendations(int userId)

8. LockService (Distributed)

- - boolean acquire(String key, long timeoutMs)
- - void release(String key)

RECOMMENDATION STRATEGY:

- **RecommendationStrategy (Interface)**
 - + List<Song> generate(int userId, List<ListeningHistory> history)
- GenreBasedStrategy
- PopularityBasedStrategy
- CollaborativeFilteringStrategy
- **RecommendationService (Context)**

REPOSITORIES:

- SongRepository

- UserRepository
 - PlaylistRepository
 - AlbumRepository
 - ArtistRepository
 - PlaybackSessionRepository
 - DownloadRepository
 - ListeningHistoryRepository
-

PATTERNS & KEY RELATIONSHIPS:

- **Association (uses)** – PlaybackSession → User, Song
 - **Association (uses)** – Playlist → User
 - **Association (uses)** – Song → Artist, Album
 - **Association (uses)** – Download → User, Song
 - **Dependency** – Services depend on repositories, CacheService, LockService
 - **Strategy Pattern** – Recommendation algorithms are interchangeable
 - **Repository Pattern** – Database access abstraction
-

Step 5: Core Use Cases & Methods

1. Play Song / Album / Playlist Use Case:

play(request) →

- Validate user existence and subscription tier
- Build playback queue based on sourceType:
 - SONG → queue = [songId]

- ALBUM → fetch songs from AlbumRepository → queue = songIds
 - PLAYLIST → fetch Playlist → queue = playlist.songIds
 - Apply shuffle if enabled (randomize queue)
 - Create or update PlaybackSession:
 - status = PLAYING
 - currentSongId = queue[0]
 - currentPosition = startPosition or 0
 - Fetch stream URL via StreamingService.getStreamUrl(currentSongId, userQuality)
 - Return PlaybackStateResponse with stream URL
 - *Note:* Client periodically calls updatePosition (every 30–60 seconds)
-

2. Stream Audio Chunk Use Case:

stream(songId, start, end) →

- Validate user access based on subscription tier
 - Check CacheService.getChunk(songId, start, end)
 - If cached: return chunk directly from cache
 - If not cached:
 - Fetch chunk from CDN or storage using byte range
 - CacheService.putChunk(songId, start, end, chunkData)
 - Return chunk
 - Set HTTP headers: Content-Type, Content-Range, Accept-Ranges
 - Return HTTP 206 (Partial Content)
-

3. Pause Playback Use Case:

pause(sessionId) →

- Fetch PlaybackSession by sessionId
 - Update PlaybackSession:
 - status = PAUSED
 - currentPosition preserved
 - lastUpdatedAt = now
 - Return PlaybackStateResponse
-

4. Skip Next Use Case:

skipNext(sessionId) →

- Fetch PlaybackSession
 - Save listening history for current song
 - Determine next song based on repeatMode:
 - OFF → move to next index, stop if end reached
 - ONE → replay same song
 - ALL → move to next index, wrap to start if needed
 - Apply shuffle if enabled
 - Update PlaybackSession(currentSongId, currentPosition=0)
 - Fetch new stream URL
 - Return PlaybackStateResponse
-

5. Create Playlist Use Case:

createPlaylist(userId, name, songIds) →

- Validate all songIds exist
- Create Playlist(userId, name, songIds, isPublic=false)

- Return Playlist
-

6. Update Playlist Use Case:

updatePlaylist(playlistId, userId, name, songIds) →

- Acquire distributed lock (playlist_lock_{playlistId})
 - Fetch Playlist by playlistId
 - Validate playlist ownership
 - Validate songs exist
 - Update Playlist(name, songIds, updatedAt=now)
 - Release lock
 - Return Playlist
-

7. Download Song Use Case:

download(userId, songId, deviceId) →

- Validate PREMIUM subscription
 - Validate device limit (max 5 devices)
 - Validate download limit (max 10,000 songs)
 - Create Download(status=PENDING)
 - Async processing:
 - Download full song from CDN/storage
 - Store in device cache
 - Update Download(status=COMPLETED, localFilePath)
 - Return Download
-

8. Get Recommendations Use Case:

getRecommendations(userId) →

- Fetch user listening history
 - Apply RecommendationStrategy.generate(userId, history)
 - Return top-N recommendations
-

9. Search Use Case:

search(query, type) →

- Search songs, artists, albums based on type
 - Aggregate results into SearchResponse
 - Return SearchResponse
-

10. Cache Eviction Use Case:

evictLRU(maxSize) →

- Check total cache size
 - If exceeded:
 - Sort cache entries by last accessed time
 - Evict least recently used chunks
-

11. Update Playback Position Use Case (Periodic):

updatePosition(sessionId, position) →

- Fetch PlaybackSession
 - Update currentPosition and lastUpdatedAt
 - Fetch song duration
 - Determine completion ($\geq 90\%$ played)
 - Create or update ListeningHistory
 - Return success
 - *Note:* Called every 30–60 seconds while playing
-

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **Repository Pattern** – data access abstraction for Song/User/Playlist/Album/Artist/PlaybackSession/Download
 - **Service Layer** – business logic separation (PlaybackService/StreamingService/CacheService/PlaylistService)
 - **Strategy Pattern** – RecommendationStrategy (GenreBased, PopularityBased, CollaborativeFiltering)
 - **RESTful API Design** – clean resource-oriented endpoints
-

OOP Principles Applied:

- **Single Responsibility** – each service focuses on one concern:
 - PlaybackService manages playback
 - StreamingService handles streaming
 - CacheService handles cache operations
- **Open/Closed** – add new recommendation strategies without modifying core logic

- **Encapsulation** – playback state transitions only via PlaybackService; cache updates only via CacheService
 - **Dependency Inversion** – services depend on interfaces (repositories, strategies, cache service)
 - **Idempotency** – playback control operations can be safely retried without side effects
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Network Interruption During Streaming:**
 - Client retries chunk request with same byte range
 - Server returns cached chunk if available
- **Cache Full:**
 - Apply LRU eviction to free space before caching new chunks
 - Evict least recently used chunks first
- **Concurrent Playlist Updates:**
 - Use distributed locks on playlist to prevent race conditions
- **Device Limit Exceeded:**
 - Validate before download; reject with clear error message
- **Download Limit Exceeded:**
 - Validate before download; suggest removing old downloads
- **Song Not Found:**
 - Return 404 with clear error; handle gracefully in UI
- **Invalid Byte Range:**
 - Validate start < end and within file size
 - Return 416 Range Not Satisfiable if invalid
- **Premium Subscription Expired:**
 - Block new downloads
 - Allow playback of already downloaded songs for grace period

- **Empty Playlist/Album:**
 - Validate before play; return error or skip gracefully
 - **Shuffle on Single Song:**
 - Shuffle has no effect; play same song
 - **Repeat Mode Edge Cases:**
 - ONE: loop current song indefinitely
 - ALL: loop entire queue
 - **Multiple Devices Playing:**
 - Each device has separate PlaybackSession; no conflict
 - **Chunk Request Out of Order:**
 - Server handles each request independently
 - Client manages buffering
 - **Cache Invalidation:**
 - When song is updated or removed, evict all cached chunks for that song
 - **Large File Streaming:**
 - Support HTTP range requests for files of any size
 - Chunk size configurable (e.g., 1MB)
 - **App Closure During Playback:**
 - Last periodic position update (every 30–60s) captures listening history
 - No special handling needed
 - **Position Update Failures:**
 - Client retries with exponential backoff
 - Missing updates are acceptable; next update will capture progress
-

Step 7.5: Streaming Protocol Explanation

Overview:

Audio streaming requires delivering audio data to clients in a way that enables playback to start quickly while the file is still being downloaded. There are several approaches:

Streaming Protocol Approaches

Approach 1: HTTP Range Requests (Selected)

- Client requests specific byte ranges of a file
- Server responds HTTP 206 Partial Content
- Client can request multiple ranges in parallel
- **Example Flow:**
 - GET /api/stream/song123?start=0&end=1048575 (Range: bytes=0-1048575)
 - Server returns first 1MB chunk
 - Client requests next chunk while playing current
 - Repeat until file complete
- **Benefits:** Simple, standard HTTP, seeking supported, works with any CDN/server
- **Limitations:** Client manages chunk buffering, no automatic quality adaptation

Approach 2: HLS (HTTP Live Streaming)

- Apple protocol, splits audio into small segments (~10s)
- Playlist (.m3u8) lists segments
- Client downloads segments sequentially
- **Benefits:** Adaptive quality, error recovery, live & on-demand
- **Limitations:** More complex, requires pre-segmenting, playlist overhead

Approach 3: DASH (Dynamic Adaptive Streaming over HTTP)

- Similar to HLS, uses XML manifest files
- Supports multiple tracks & quality levels
- **Benefits:** Flexible, feature-rich, industry standard

- **Limitations:** Complex, requires infrastructure

Implementation Details for HTTP Range Requests:

- **Chunk Size:** 1MB (1048576 bytes), configurable
- **Cache Key Format:** "chunk_{songId}_{start}_{end}"
- **HTTP Status:** 206 Partial Content
- **Headers:** Content-Range, Accept-Ranges: bytes
- **Client Behavior:** request chunks sequentially or in parallel for buffering
- **Prefetching:** Client can request next chunk while current plays

Selected Approach: HTTP Range Requests (Approach 1)

For interview scope, we use **HTTP Range Requests** because:

- **Simplicity:** Easy to understand and implement
- **Interview-friendly:** Demonstrates understanding of HTTP protocol
- **Sufficient:** Meets requirements for chunk-based streaming
- **Flexible:** Supports seeking and progressive playback
- **Standard:** Works with any HTTP client/server

Implementation Details:

- **Chunk Size:** 1MB (1048576 bytes) - configurable
 - **Cache Key Format:** "chunk_{songId}_{start}_{end}"
 - **HTTP Status:** 206 Partial Content for range requests
 - **Headers:** Content-Range, Accept-Ranges: bytes
 - **Client Behavior:** Request chunks sequentially or in parallel for buffering
 - **Prefetching:** Client can request next chunk while current chunk is playing
-

Step 9: Future Requirements

Future Functional Requirements:

- Social features: Follow artists, share playlists, collaborative playlists
 - Advanced recommendations: Machine learning-based personalization
 - Podcast support: Audio content beyond music
 - Live streaming: Real-time audio streaming events
 - Lyrics display: Synchronized lyrics during playback
 - Radio stations: Algorithmic radio based on seed songs
 - Crossfade: Smooth transitions between songs
 - Equalizer: Audio effects and customization
 - Playlist folders: Organize playlists into folders
 - Smart playlists: Auto-generated playlists based on rules
 - Offline sync: Sync playback state across devices
 - Voice commands: Control playback via voice
 - Integration with smart speakers: Alexa, Google Home support
 - Analytics dashboard: Listening statistics and insights
 - Artist verification: Verified artist badges and profiles
-

Summary

This design targets a 60–120 minute interview with 5 core flows:

- **Search:** GET /api/search (find songs, albums, artists, playlists)
- **Play:** POST /api/playback/play (stream songs, albums, playlists uniformly)
- **Stream:** GET /api/stream/{songId} (chunk-based HTTP range requests)

- **Playlist Management:** CRUD operations with concurrent update handling
 - **Download:** POST /api/downloads (offline playback with device/limit validation)
-

Additional Features:

- **Position Tracking:** POST /api/playback/position (periodic updates every 30-60s for listening history)
- **Recommendations:** GET /api/recommendations (based on listening history)

Highlights:

- HTTP Range Requests for chunk-based streaming (STEP-7.5)
- Caching with LRU eviction policy (evicts least recently used chunks when cache is full)
- Playlist management with distributed locking for concurrency
- Strategy Pattern for recommendations
- Repository/Service layering with clear separation of concerns
- Multi-device support with download limits and device tracking
- Periodic position updates for listening history tracking (handles app closure gracefully)
- Clear edge-case coverage suitable for production conversations